LAB DESIGN AND IMPLEMENTATION OF A SCALABLE INFORMATION MANAGEMENT SYSTEM FOR A RETAIL BUSINESS

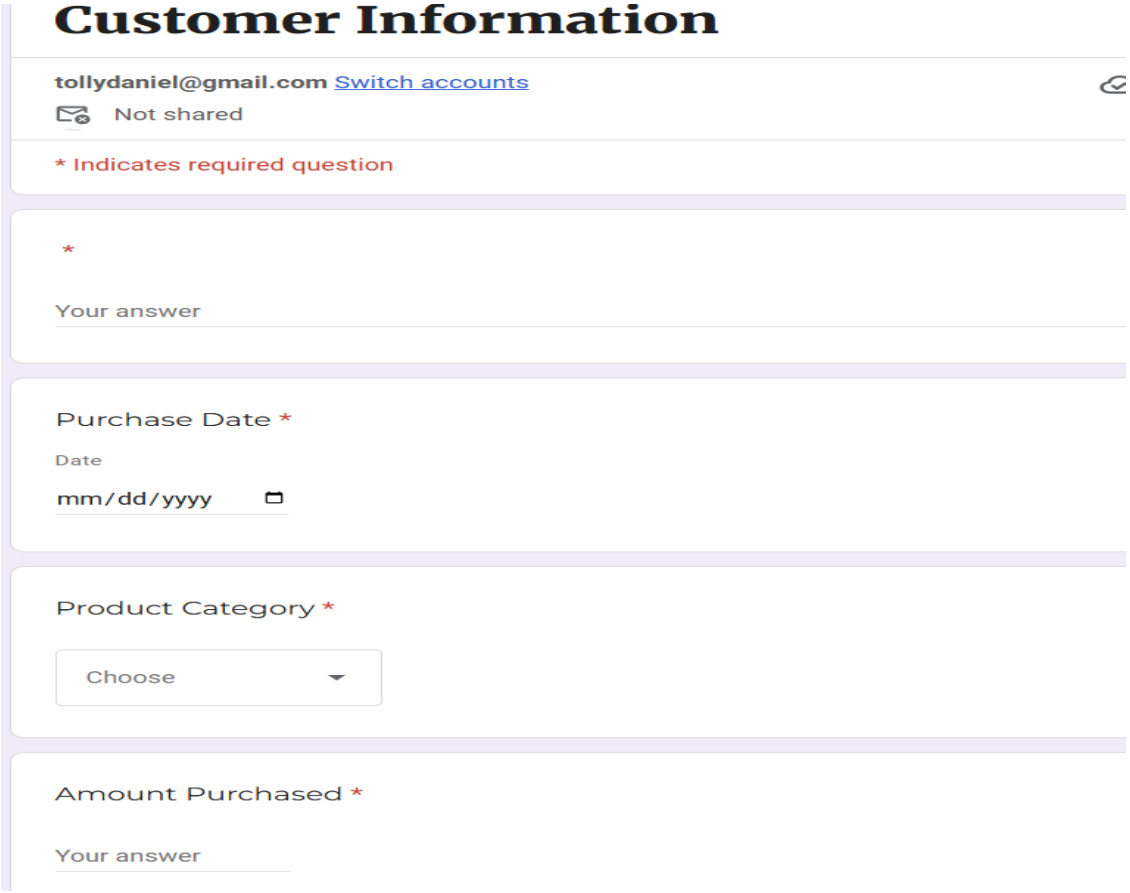
Team Report – DTS 304 Data Management I

TEAM MEMBERS WITH MATRICULATION NUMBERS

S/N	Name	Matriculation Number
1	Ajidahun Tolulope Daniel	2025/A/DSC/0044
2	Jane Adaola Eso	2024/B/DSC/0098
3	Aderibigbe Otunde	2023/C/DSC/0153
4	Olowookere Oluwaseyifunmi Paul	2024/B/DSC/0106
5	Akande Rukayat Adeola	2025/A/DSC/0050

TASK 1: **DESIGN A CUSTOMER PURCHASE FORM USING GOOGLE FORMS.**

The team designed a google form for users to fill where products were selected ,this had 31 datasets from 31 customers taking in their email and the customer ID,the product category as a List of Values(LOV) and the amount in figures . The dataset were captured into a CSV file which would be later loaded into a table.



TASK 2:DATA INDEXING AND RETRIEVAL (WEEK 3 CONCEPTS)

Using PostgresSQL DB , a table **PURCHASES** was created created in a schema **RETAIL_DB** with the primary index created on Customer_Id and Secondary index created on product_category and a non clustered index was created on Purchase_date ,

```
CREATE INDEX idx_customer_id

ON purchases(customer_id);

CREATE INDEX idx_product_category

ON purchases(product_category);


CREATE INDEX idx_purchase_date

ON purchases(purchase_date);
```

To retrieve the products in a date ranger specifically from **'2026-03-01' to '2026-03-15'** using the date format **YYYY-MM-DD.** The below query was ran

```
SELECT *

FROM purchases

WHERE purchase_date BETWEEN '2026-03-01' AND '2026-03-15';
```

To retrieve products under the category **Electronics** the below query was ran

```
SELECT *

FROM purchases

WHERE upper (product_category) = upper('Electronics')

ORDER BY purchase_date;
```

TASK 3: SEARCH STIMULATION

The team implemented a binary search algorithm .

TASK3A: IMPLEMENT BINARY SEARCH USING PYTHON

The goal of this task was to **implement a binary search algorithm in Python** to search for a specific product_id from a PostgreSQL database table. The binary search provides an efficient way to locate an item in a **sorted dataset**, achieving **O(log n)** time complexity.

This was achieved using the following python libraries :

psycopg2 – psycopg is a PostgresSQL database adapter for the Python programming language,

time - This module provides various functions to manipulate time values,

dotenv - Load environment variables from a file (usually .env) into Python ,this enabled us not to hardcode the DB credentials used to connect to the PostgresSQL DB as it is a security risk,

OS- to get the environmental variable name and read the file which the DB credentials sits in.

Steps taken are highlighted below:

- The database credentials were stored in a .env file (DB_CONFIG.env) to avoid hardcoding sensitive information.
- The dotenv module was used to load these credentials:

```
load_dotenv(r'C:\Users\USER\DTS304\DB_CONFIG.env')
DB_CONFIG = {
    "database": os.getenv("DB_NAME"),
    "user": os.getenv("DB_USER"),
    "password": os.getenv("DB_PASSWORD"),
    "host": os.getenv("DB_HOST"),
    "port": os.getenv("DB_PORT")
}
```

- Used psycopg2.connect() with the loaded credentials to establish a connection.
- Created a cursor cur to execute SQL queries:

```
conn = psycopg2.connect(**DB_CONFIG)
cur = conn.cursor()
```

- Queried the purchases table to retrieve all product_id

```
cur.execute("SELECT product_id FROM purchases ORDER BY product_id;")
rows = cur.fetchall()
```

- Sorted the product IDs in ascending order (required for binary search).

```
product_ids = [row[0] for row in rows if row[0] is not None]
```

- Defined a binary_search function that takes a **sorted list** and a **target value**, The function works by repeatedly dividing the search interval in half until the target is found or the interval is empty.

```
Filtered out any NULL values to avoid errors during the search.
def binary_search(data, target):
    left = 0
    right = len(data) - 1
    while left <= right:
        mid = (left + right) // 2
        if data[mid] == target:
            return mid
        elif data[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

- The user enters a product_id to search and the script prints out if the product id was found and the index in the sorted list.

```
target = int(input("Enter product ID to search: "))
index = binary_search(product_ids, target)
```

- Then the cursor and connection is closed , this enable resources allocated to the system to be freed for another operation.

```
cur.close()

conn.close()
```

TASK3B :TO STIMULATE A SEQUENTIAL SEARCH ON UNINDEXED DATA TO COMPARE PERFORMANCE.

The aim of this task was to simulate a sequential search on unindexed data and compare its performance with a binary search on sorted data.

- Sequential search checks each element one by one in the order retrieved from the database.
- Binary search is applied to a sorted list and uses a divide-and-conquer approach to locate the target efficiently.

This demonstrates the performance difference between unindexed (unsorted) search and indexed search techniques.

Same libraries , tools and DB configuration files were used. were used (as Binary search implementation).

- A connection to the Database was established and a cursor was created to execute SQL queries:
  ```
  conn = psycopg2.connect(**DB_CONFIG)
  cur = conn.cursor()
  ```
- All product IDs from the purchases table was retrieved
  ```
  cur.execute("SELECT product_id FROM purchases;")
  rows = cur.fetchall()
  product_ids = [row[0] for row in rows if row[0] is not None]
  ```
  Here the product_id is unsorted
- Created a **sorted copy** for binary search:
  ```
  sorted_product_ids = sorted(product_ids)
  ```
- Implemented a **linear search** over the unsorted list:

```
seq_index = -1

for i, value in enumerate(product_ids):

    if value == target:

        seq_index = i

        break
```

- Implemented standard **binary search** for sorted data and Measured the execution time similarly to sequential search.

```
def binary_search(data, target):
    left = 0
    right = len(data) - 1
    while left <= right:
        mid = (left + right) // 2
        if data[mid] == target:
            return mid
        elif data[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

- The program **prompts the user** to enter a product ID. Both searches were performed on the entered target, and results were displayed:

```
Enter product ID to search: 4
Sequential Search: Product ID 4 found at index 16
Sequential Search Time: 0.000012 seconds
Binary Search: Product ID 4 found at index 19
Binary Search Time: 0.000006 seconds
```

- Cursor and connection were closed to free resources:

```
cur.close()
conn.close()
```

SUMMARY AND OBSERVATIONS

  - **Sequential search** works on **unsorted/unindexed data**, but its performance degrades linearly with dataset size.
  - **Binary search** works on **sorted data** and is much faster, achieving **O(log n)** complexity.

- This simulation clearly shows the **performance difference** between unindexed and indexed search strategies:
- Sequential search takes longer as the dataset grows.
- Binary search is more efficient on sorted or indexed datasets.

Using Python and PostgreSQL together demonstrates how database data can be **queried, processed, and analyzed efficiently** using different algorithms.